

10. TEST EXECUTION

The Test Execution phase is one of the most important stages of the Software Testing Life Cycle (STLC). During this phase, all prepared test cases are executed against the target application to verify functionality, identify vulnerabilities, and validate security controls.

In this project, test execution was performed on the **Damn Vulnerable Web Application (DVWA)** using Selenium WebDriver, TestNG, Maven, and OWASP ZAP. The execution process included Login Testing, SQL Injection Testing, Cross-Site Scripting (XSS) Testing, Security Scanning, and Report Generation.

The primary objective of test execution was to identify security vulnerabilities, evaluate application behavior against malicious inputs, and generate detailed execution reports.

10.1 Login Test Execution

The Login Test Execution was performed to verify the authentication functionality of DVWA. Successful login is required before accessing other application modules.

Objective

The objective of login testing was to validate that the application correctly authenticates valid users and redirects them to the DVWA Home Page.

Test Scenario

Validate successful login using valid credentials.

Test Data

Parameter	Value
Username	admin
Password	password

Execution Steps

1. Launch Chrome Browser.
2. Navigate to DVWA Login Page.
3. Enter valid username.
4. Enter valid password.
5. Click the Login button.
6. Observe application behavior.

Expected Result

- User should be authenticated successfully.
- Application should redirect to DVWA Home Page.
- User session should be created.

Actual Result

- User authentication completed successfully.
- User was redirected to DVWA Home Page.
- Session was established successfully.

Status

PASS

Observation

The login functionality operated as expected and allowed access to application modules required for further testing.

10.2 SQL Injection Execution

SQL Injection testing was performed to determine whether the application properly validates user input before interacting with the database.

DVWA intentionally contains SQL Injection vulnerabilities, making it suitable for security testing practice.

Objective

To identify SQL Injection vulnerabilities and evaluate application behavior when malicious SQL payloads are submitted.

Test Scenario

Validate application behavior against multiple SQL Injection payloads.

Test Payloads Used

Payload 1

```
' OR '1'='1
```

Payload 2

```
1' OR '1'='1' --
```

Payload 3

```
admin' #
```

Payload 4

```
' UNION SELECT user,password FROM users --
```

Execution Steps

1. Navigate to SQL Injection Module.
2. Enter SQL payload.
3. Submit request.
4. Observe response.
5. Record findings.

Expected Result

- Application should validate input.
- Malicious SQL queries should be rejected.
- Unauthorized database access should be prevented.

Actual Result

- Application accepted malicious SQL payloads.
- Database records were exposed.
- Authentication bypass was possible in some cases.

Status

FAIL

Vulnerability Identified

SQL Injection

Severity

Critical

Observation

The application failed to validate malicious SQL input, resulting in unauthorized access to database information.

10.3 XSS Execution

Cross-Site Scripting (XSS) testing was performed to determine whether the application properly sanitizes user-supplied input before displaying it within web pages.

Objective

To identify Cross-Site Scripting vulnerabilities and evaluate application protection mechanisms.

Test Scenario

Validate application behavior against multiple XSS payloads.

Test Payloads Used

Payload 1

```
<script>alert('XSS')</script>
```

Payload 2

```
<img src=x onerror=alert('XSS')>
```

Payload 3

```
<svg onload=alert('XSS')>
```

Payload 4

```
<script>document.cookie</script>
```

Execution Steps

1. Navigate to XSS Module.
2. Enter XSS payload.
3. Submit request.
4. Observe browser response.
5. Record findings.

Expected Result

- User input should be sanitized.
- JavaScript execution should be prevented.
- Malicious content should not be rendered.

Actual Result

- JavaScript payload executed successfully.
- Alert messages appeared in browser.
- Input sanitization was not implemented effectively.

Status

FAIL

Vulnerability Identified

Cross-Site Scripting (XSS)

Severity

Critical

Observation

The application was vulnerable to reflected XSS attacks and allowed execution of malicious client-side scripts.

10.4 Security Scan Execution

Security scanning was performed using OWASP ZAP to automatically identify vulnerabilities within the application.

The scanning process consisted of Spider Scan and Active Scan activities.

Spider Scan Execution

Objective

To discover all accessible pages, URLs, forms, and parameters within the application.

Execution Steps

1. Launch OWASP ZAP.
2. Configure target URL.
3. Start Spider Scan.
4. Monitor discovered pages.

Target URL

`http://localhost/DVWA`

Expected Result

- Application pages should be discovered.
- URLs should be listed within OWASP ZAP.

Actual Result

- Multiple URLs and pages were discovered successfully.
- Application structure was mapped correctly.

Status

PASS

Active Scan Execution

Objective

To identify security vulnerabilities within discovered application pages.

Execution Steps

1. Select discovered target.
2. Start Active Scan.
3. Wait for scan completion.
4. Review generated alerts.

Expected Result

- Security vulnerabilities should be identified.
- Risk classifications should be assigned.

Actual Result

- Multiple vulnerabilities were detected.
- Risk levels were assigned successfully.
- Alerts were generated and categorized.

Status

PASS

Sample Vulnerabilities Detected

- SQL Injection
 - Cross-Site Scripting (XSS)
 - Missing Security Headers
 - Information Disclosure
 - Cookie Without Secure Flag
 - Cookie Without HttpOnly Flag
-

10.5 TestNG Execution

TestNG was used as the test execution framework for running automated test scripts.

Objective

To execute automation test cases and generate execution reports.

Execution Method

The test suite was executed using the TestNG XML configuration file.

Sample Command

```
mvn test
```

OR

Run:

```
testng.xml
```

from Eclipse.

Activities Performed

- Test Case Execution
- Result Collection
- Failure Tracking
- Report Generation

Expected Result

- All test cases should execute successfully.
- Reports should be generated automatically.

Actual Result

- Test cases executed successfully.
- Pass and Fail statuses were recorded.
- Reports were generated automatically.

Status

PASS

Sample Execution Summary

Metric	Count
Total Test Cases	20
Passed	15
Failed	5
Skipped	0

10.6 Execution Screenshots

Screenshots were captured throughout test execution to provide evidence of application behavior, vulnerabilities, and report generation.

Login Execution Screenshot

Screenshot captured after successful login to DVWA.

Evidence includes:

- Username entered
 - Successful authentication
 - DVWA Home Page displayed
-

SQL Injection Screenshot

Screenshot captured after SQL payload execution.

Evidence includes:

- SQL payload submitted
 - Unauthorized data displayed
 - Vulnerability confirmation
-

XSS Screenshot

Screenshot captured after XSS payload execution.

Evidence includes:

- XSS payload entered
 - Alert popup displayed
 - Successful script execution
-

OWASP ZAP Scan Screenshot

Screenshot captured after Active Scan completion.

Evidence includes:

- Scan progress
 - Vulnerability alerts
 - Risk severity classification
-

TestNG Report Screenshot

Screenshot captured from generated TestNG report.

Evidence includes:

- Total test count
 - Passed test count
 - Failed test count
 - Execution summary
-

Extent Report Screenshot

Screenshot captured from generated Extent Report.

Evidence includes:

- Detailed execution logs
- Test status
- Screenshots attached to failed tests

Test Execution Summary

The Test Execution phase successfully validated both functional and security aspects of the DVWA application. Login functionality operated correctly, while SQL Injection and Cross-Site Scripting vulnerabilities were successfully identified using automated test cases. OWASP ZAP scans detected multiple security weaknesses and generated detailed vulnerability reports. TestNG and Extent Reports provided execution statistics and evidence of testing activities.

The execution results demonstrate the effectiveness of combining Selenium WebDriver, TestNG, Maven, and OWASP ZAP to automate web application security testing and vulnerability assessment.